

COL775-A2 Submission Guidelines

These instructions define the required submission format for the Assignment 2 auto-grader. Follow the file names, command-line arguments, and output formats exactly. If the auto-grader cannot import your code, load your checkpoints, run the required scripts, or parse your outputs, the corresponding automated marks may be lost.

1. Submit One Zip File

Submit one zip file named **TeamName_A2.zip**

Unzipping it must create exactly one top-level directory: **TeamName_A2/**

Use a short team name containing only letters, digits, and underscores. Do not use spaces in the zip name or the top-level directory name instead use “_”.

Any one team member may submit the zip file on Moodle. Do not submit the dataset.

2. Required Directory Layout

Your submission must have the following structure:

```
TeamName_A2/ |-- PartA/ | |-- inference_linear_probing.py | |-- inference_retrieval.py | |-- <all other .py files needed by the two inference scripts> |
-- PartB/ | |-- inference.py | |-- <all other files needed for Part B, if applicable> | |-- PartC/ | |-- inference.py | |-- <all other files needed for Part C, if applicable> | |-- report.pdf | |-- TeamInfo.txt
```

Include all source files needed to instantiate your CLIP model, DINO student, DINO teacher, tokenizer, data loading utilities, and linear probes.

report.pdf must be a single report for the full assignment. It must include the metric table, t-SNE plots, retrieval examples, and brief observations requested in the assignment statement.

TeamInfo.txt must use this format:

```
TeamName Name1, EntryNumber1 Name2, EntryNumber2 Name3, EntryNumber3 http
s://drive.google.com/file/d/0B1HXnM1lBuoqMzVhZjcwNTAtZWl5OS00ZDg3LWEyMzktNzZ
mYWY2Y2NhNWQx/view?hl=en&resourcekey=0-5DqnTtXPFvySM1WstuAYdA
```

The last line of **TeamInfo.txt must be a valid link to the google drive containing all the model weights as described in Section 7. You should also place this link in your report.**

3. Environment Assumptions

Your code will be evaluated with Python 3.10 and PyTorch 2.5.0+cu11.8. If you use additional packages, list them in **requirements.txt** in the submission root. Keep dependencies minimal and avoid packages that require downloading files at inference time.

Inference scripts must not train models, download datasets, download pretrained weights, or require internet access. They should only load files already present in the submitted zip, especially the part-specific directory passed through `--model_dir`, and read the input passed through `--data_path`.

4. Part A Scripts

Place both scripts directly inside `PartA/`:

```
PartA/inference_linear_probing.py PartA/inference_retrieval.py
```

The script will be run from inside `TeamName_A2/PartA/`. The autograder will use hidden input JSON files with the same schema as the released files. Do not hard-code the number of examples, file names, labels, or local dataset paths.

4.1 Linear Probing Inference

The autograder will run:

```
python inference_linear_probing.py \ --model_type <clip|dino_student|dino_teacher> \ --model_dir <path_to_partA_models_directory> \ --pooling_type <cls|gap> \ --probe_task <count|color> \ --data_path <path_to_probe_json> \ --output_file <path_to_output_json>
```

Argument meanings:

- `-model_type`: which frozen vision encoder to use.
- `-model_dir`: path to the Part A model directory containing all backbone checkpoints, tokenizer files, configs, and linear-probe weights needed by your script.
- `-pooling_type`: which pooling to use either `cls` or `gap` as described in the assignment.
- `-probe_task`: `count` for object-count prediction, or `color` for multi-label color-set prediction.
- `-data_path`: path to a probe JSON file such as `clevr_count_val.json` or `clevr_colors_val.json`.
- `-output_file`: exact file path where your script must save predictions.

The input probe JSON has this structure:

```
{ "metadata": { "task": "count or colors", "split": "val", "num_examples": 15000 }, "examples": [ { "image_filename": "CLEVR_val_000000.png", "image_path": "/absolute/path/to/CLEVR_val_000000.png" } ] }
```

For released validation files, labels are included. Hidden test files may omit labels. Your inference code must not depend on label fields being present.

4.1.1 Count Output Format

For `--probe_task count`, write a JSON object mapping image filename to predicted object count:

```
{ "CLEVR_val_000000.png": 5, "CLEVR_val_000001.png": 10 }
```

Values must be JSON integers, not strings.

4.1.2 Color Output Format

For `--probe_task color`, write a JSON object mapping image filename to the predicted set of colors present in the image:

```
{ "CLEVR_val_000000.png": ["brown", "gray", "green", "purple"], "CLEVR_val_000001.png": ["blue", "cyan", "gray"] }
```

4.2 CLIP Retrieval Inference

Retrieval is evaluated only for CLIP, because DINO has no text encoder in this assignment.

The auto-grader will run:

```
python inference_retrieval.py \ --model_type clip \ --model_dir <path_to_partA_models_directory> \ --retrieval_task <i2t|t2i> \ --data_path <path_to_caption_json> \ --output_file <path_to_output_json>
```

Argument meanings:

- `-model_type`: must be `clip`.
- `-model_dir`: path to the Part A model directory containing the trained CLIP checkpoint, tokenizer/vocabulary files, and any configs needed to reconstruct the CLIP image and text encoders.
- `-retrieval_task`: `i2t` for image-to-text retrieval, or `t2i` for text-to-image retrieval.
- `-data_path`: path to a caption JSON file such as `clevr_val_captions.json`.
- `-output_file`: exact file path where your script must save retrieval results.

The input caption JSON is a list of examples in the released Part A data:

```
[ { "image_filename": "CLEVR_new_val_000000.png", "caption": "An image with 10 objects: ..." "image_path": "/absolute/path/to/CLEVR_val_000000.png" } ]
```

Hidden test file will include the extra absolute `image_path` field, you must use it to load the image.

Compute cosine similarity between L2-normalized embeddings. Return the top 3 matches for every query, ordered from most similar to least similar.

4.2.1 Image-to-Text Output Format

For `--retrieval_task i2t`, write a JSON object mapping each query image filename to a list of the top 3 retrieved captions:

```
{ "CLEVR_new_val_000000.png": [ "An image with 10 objects: ...", "An image with 9 objects: ...", "An image with 10 objects: ..." ] }
```

4.2.2 Text-to-Image Output Format

For `--retrieval_task t2i`, write a JSON object mapping each query caption to a list of the top 3 retrieved image filenames:

```
{ "An image with 10 objects: ...": [ "CLEVR_new_val_000000.png", "CLEVR_new_val_000142.png", "CLEVR_new_val_000381.png" ] }
```

No need to worry about duplicate captions in the input data for t2i retrieval task

5. Part B: Vision-Language Modeling

Place the Part B inference script directly inside `PartB/`:

The script will be run from inside `TeamName_A2/PartB/`. It must load the frozen vision encoder, projector, LoRA adapters, tokenizer/config files, and any other Part B artifacts from `--model_dir`, run inference on CLEVR questions, and write the predictions to the output JSON path supplied by the autograder.

The autograder will run:

```
python inference.py \ --model_dir <path_to_partB_models_directory> \ --data_path <path_to_CLEVR_questions_json> \ --output_file <path_to_save_qa_predictions_json>
```

Argument meanings:

- `-model_dir`: path to the Part B model directory containing everything needed for VLM inference, including the frozen vision encoder checkpoint used by the VLM, MLP projector weights, LoRA adapters, tokenizer files, and configs.
- `-data_path`: path to a CLEVR questions JSON file such as `CLEVR_val_questions.json`.
- `-output_file`: exact file path where your script must save the QA prediction JSON.

The CLEVR questions JSON has this structure:

```
{ "info": {}, "questions": [ { "question_index": 0, "image_filename": "CLEVR_val_000000.png", "split": "val", "question": "Are there any other things that are the same shape as the big metallic object?" "image_path": "/absolute/path/to/CLEVR_val_000000.png" } ] }
```

Released validation files include `answer` and `program` fields. Hidden test files will omit these fields. Your inference code must not depend on ground-truth answers, programs, or explanations being present.

each sample will contain the absolute image path under `image_path` field as shown above. You must load images from it.

5.1 Part B Output Format

Write a single JSON object to `--output_file`. Keys must be question indices as strings. Values must be JSON objects with exactly two fields:

```
{ "0": { "reasoning": "The image contains a large metallic object. I compare its shape with the other visible objects and do not find another object of the same shape.", "answer": "no" }, "1": { "reasoning": "The scene contains three cyan cylinders and two blue cubes, so the requested count is five.", "answer": "5" } }
```

Output requirements:

- Use `str(question_index)` as the key when `question_index` is present.
- `reasoning` and `answer` must be a string.
- Write predictions for every input question and no extra questions.

As mentioned on piazza (https://piazza.com/class/mkjwhjzdk51t/post/24_f2), `Qwen/Qwen3-4B-Instruct-2507` will be cached locally, no need to mention it's loading path.

Do not download Qwen, LoRA adapters, tokenizer files, or any other weights during inference. If your inference code needs tokenizer/config files that are not already available in the evaluation environment, include them in `PartB/models/`.

6. Part C: Generative Modeling

Place the Part C inference script directly inside `PartC/`:

The script will be run from inside `TeamName_A2/PartC/`. It must support two tasks:

- `reconstruct`: reconstruct validation images using the trained VAE.
- `generate`: generate images from captions using the trained LDM/U-Net and VAE decoder.

The autograder will run:

```
python inference.py \ --model_dir <path_to_partC_models_directory> \ --task <reconstruct|generate> \ --data_path <path_to_input_directory_or_json> \ --output_dir <path_to_save_images_directory>
```

Argument meanings:

- `-model_dir`: path to the Part C model directory containing everything needed for VAE reconstruction and LDM generation, including VAE weights, LDM/U-Net weights, latent normalization statistics, text-encoder/tokenizer files, and configs.
- `-task`: `reconstruct` for VAE reconstruction, or `generate` for text-conditioned image generation.
- `--data_path`: for `reconstruct`, path to a directory containing input images; for `generate`, path to a caption JSON file such as `clevr_val_captions.json`.
- `-output_dir`: directory where output images must be saved. No need to create it, it will exist before your code is ran.

6.1 Reconstruct Task Input and Output

For `--task reconstruct`, `--data_path` is a directory containing validation images:

```
<data_path>/ |-- CLEVR_val_000000.png |-- CLEVR_val_000001.png |-- ...
```

Your script must reconstruct every `.png`, `.jpg`, or `.jpeg` image directly inside that directory. Save each reconstruction using the original filename:

```
<output_dir>/ |-- CLEVR_val_000000.png |-- CLEVR_val_000001.png |-- ...
```

Output image requirements:

- Save RGB images in PNG format.
- Use the same filename stem as the input image.
- Final saved images should be `128 x 128` pixels, matching the Part C assignment setup.

6.2 Generate Task Input and Output

For `--task generate`, `--data_path` is a caption JSON file. The released caption JSON has this structure:

```
[ { "image_index": 0, "image_filename": "CLEVR_val_000000.png", "split": "val", "caption": "An image with 5 objects: ..." } ]
```

Your script must generate one image for every caption in the input JSON. Save images directly inside `--output_dir` using the `image_filename`

```
<output_dir>/ |-- CLEVR_val_000000.png |-- CLEVR_val_000001.png |-- CLEVR_val_000002.png |-- ...
```

Output image requirements:

- Save RGB images in PNG format.
- Each generated image must be `128 x 128` pixels.
- The number of output images must exactly equal the number of caption examples.

6.3 Part C Checkpoint Organization

Recommended organization:

```
PartC/ |-- inference.py |-- checkpoints/ | |-- vae.pth | |-- ldm.pth | |-- latent_stats.pth | |-- text_encoder_or_tokenizer_files_if_needed/ |-- <all Python modules needed by inference.py>
```

Do not download CLIP text encoder files, tokenizer files, or other model weights during inference. If your implementation depends on frozen text-encoder files that are not guaranteed to exist in the environment, include them with the external weights.

7. Model Submission Rule

You must upload a single .zip file to google-drive with the same convention as code submission. Unzipping it should create. Each part has exactly one model directory:

```
TeamName_A2/ |-- PartA/ |-- All model weight files required by PartA inference.py scripts |-- PartB/ |-- ... |-- PartC/ |-- ...
```

The autograder will pass the relevant directory through `--model_dir`. The internal filenames are up to each team, but loading must be fully self-contained in the submitted inference script.

Include every inference-time artifact needed by that part inside its directory:

- `.pth`, `.pt`, `.bin`, or adapter checkpoint files,
- tokenizer files and vocabularies,
- architecture/config JSON files,
- latent mean/std statistics for Part C,
- any small metadata file needed to map `model_type` or `probe_task` to the correct saved weights.

8. Global Failure Cases to Avoid

- Do not submit datasets, generated reconstructions, generated samples, or large caches in the zip file.
- Do not hard-code absolute paths from your machine.
- Do not require internet access during inference.
- Do not change script names, argument names, output file formats, or output image naming conventions.
- Do not assume labels, answers, programs, or explanations are present in hidden test files.
- Do not write outputs anywhere except the path supplied by `-output_file` or `-output_dir`.
- Do not require manual edits to checkpoint paths or config files after unzipping.
- Do not require the autograder to know your checkpoint filenames; load them from `-model_dir`.

